

Glossaire HTML / CSS / Bootstrap

RÉFÉRENCE RAPIDE

Toutes les notions techniques rencontrées dans les 19 chapitres du cours, classées par thème puis indexées alphabétiquement en fin de document. Définitions courtes en tableau, approfondissements pour les notions critiques.

Comment utiliser ce glossaire

- Tu cherches un terme précis ? Saute directement à l'**index alphabétique** en fin de document.
- Tu reviens sur une notion d'un chapitre ? Va dans la **section thématique** correspondante.
- Tu veux creuser ? Chaque entrée détaillée renvoie au chapitre CRS qui explique la notion en profondeur.

Sommaire

- **A. HTML sémantique**
- **B. CSS — fondamentaux**
- **C. Cascade et héritage**
- **D. Box-model et layout**
- **E. Flexbox et Grid**
- **F. Responsive**
- **G. Bootstrap**
- **Index alphabétique**
- **Pour aller plus loin**

A. HTML sémantique

Terme	Définition courte	Chapitre
Balise	Marqueur HTML délimitant un élément, ouvrante <code><tag></code> et fermante <code></tag></code> (sauf balises auto-fermantes).	II
Élément	Couple balise ouvrante + contenu + balise fermante, formant un nœud du DOM.	II
Attribut	Information complémentaire ajoutée à une balise ouvrante (<code></code>), sous forme <code>nom="valeur"</code> .	III
Balise sémantique	Balise qui décrit la nature du contenu (<code><header></code> , <code><article></code> , <code><nav></code>) plutôt que sa présentation.	II
Accessibilité	Capacité d'une page à être utilisée par tous, y compris avec lecteur d'écran ou navigation clavier. Cible WCAG/RGAA.	II, III
ARIA	<i>Accessible Rich Internet Applications</i> — attributs (<code>aria-label</code> , <code>role</code>) qui complètent l'accessibilité quand le HTML sémantique ne suffit pas.	III
Validation native	Contraintes HTML5 sur les <code><input></code> (<code>required</code> , <code>pattern</code> , <code>type="email"</code>) qui valident côté navigateur sans JavaScript.	IV

Balise sémantique (détail)

Le HTML5 fournit des balises qui décrivent le **rôle** du contenu : `<header>` , `<nav>` , `<main>` , `<article>` , `<section>` , `<aside>` , `<footer>` . Utiliser ces balises plutôt que des `<div>` génériques améliore :

- **Accessibilité** : un lecteur d'écran peut sauter directement à `<main>` ou lister les `<nav>` .
- **SEO** : les moteurs de recherche comprennent la structure.
- **Maintenance** : le HTML est lisible sans inspecter les classes CSS.

Exemple :

```
<header>
  <h1>Mon site</h1>
  <nav>...</nav>
</header>
<main>
  <article>
    <h2>Titre d'article</h2>
    <p>Contenu...</p>
  </article>
</main>
<footer>© 2026</footer>
```

⚠ Piège

Une page doit contenir **un seul** `<main>` et un seul `<h1>`. Plusieurs `<h1>` cassent la hiérarchie pour les lecteurs d'écran.

Voir aussi : [CRS/02_bases-bonnes-pratiques.md](#).

Attribut (détail)

Un attribut prend la forme `nom="valeur"` à l'intérieur d'une balise ouvrante. Certains sont **globaux** (applicables à toute balise : `id`, `class`, `lang`, `title`), d'autres **spécifiques** (`src` sur `<img>`, `href` sur `<a>`).

Quelques attributs essentiels :

- `id` : identifiant unique dans la page.
- `class` : permet d'appliquer des styles CSS (plusieurs classes séparées par espace).
- `alt` : texte alternatif sur `<img>` (obligatoire pour l'accessibilité).
- `href` : URL cible sur `<a>`.
- `rel="noopener noreferrer"` : sécurité sur les liens externes `target="_blank"`.
- `lang="fr"` : langue du document (sur `<html>`).

⚠ Piège

`alt=""` (vide) est valide et signifie "image décorative" — différent d'une `alt` manquante qui est une erreur d'accessibilité.

Voir aussi : [CRS/03_attributs.md](#).

B. CSS — fondamentaux

Terme	Définition courte	Chapitre
Règle CSS	Bloc <code>sélecteur { propriété: valeur; }</code> qui applique des styles aux éléments ciblés.	VI
Sélecteur	Expression qui désigne les éléments HTML auxquels une règle s'applique (type, classe, ID, attribut, combineur).	IX
Classe	Sélecteur préfixé par <code>.</code> (<code>.btn</code>) ciblant tous les éléments ayant cette classe en attribut <code>class</code> .	IX
ID	Sélecteur préfixé par <code>#</code> (<code>#header</code>) ciblant l'élément unique ayant cet <code>id</code> . À éviter en CSS pour le styling.	IX
Pseudo-classe	État d'un élément ciblé par <code>:</code> (<code>:hover</code> , <code>:focus</code> , <code>:nth-child(odd)</code>).	IX
Pseudo-élément	Sous-partie virtuelle d'un élément ciblée par <code>::</code> (<code>::before</code> , <code>::after</code>).	IX
Combineur	Opérateur entre sélecteurs : descendant (espace), enfant direct (<code>></code>), frère adjacent (<code>+</code>), frère général (<code>~</code>).	IX
Propriété	Aspect stylistique modifiable (<code>color</code> , <code>font-size</code> , <code>display</code>).	VIII
Valeur	Donnée affectée à une propriété (<code>red</code> , <code>1.5rem</code> , <code>flex</code>).	VIII
Déclaration	Couple <code>propriété: valeur;</code> à l'intérieur d'une règle.	VIII

Sélecteur (détail)

Un sélecteur peut être simple ou composé. Catégories :

Catégorie	Exemple	Cible
Type	<code>p</code>	Tous les <code><p></code>
Classe	<code>.alert</code>	Éléments avec <code>class="alert"</code>
ID	<code>#header</code>	L'élément avec <code>id="header"</code>
Attribut	<code>[type="email"]</code>	Éléments avec <code>type="email"</code>
Pseudo-classe	<code>a:hover</code>	Liens survolés
Pseudo-élément	<code>p::first-line</code>	Première ligne d'un <code><p></code>
Universel	<code>*</code>	Tous les éléments

Combinés via combineurs :

```
nav a          /* tous les <a> à l'intérieur d'un <nav> */  
.btn > .icon    /* <.icon> enfant DIRECT d'un <.btn> */  
h2 + p         /* <p> immédiatement après un <h2> */  
h2 ~ p         /* tous les <p> frères suivants d'un <h2> */
```

♦ Bonne pratique

Préférer les **classes** aux ID en CSS : réutilisables, moins de spécificité, plus flexibles.

Voir aussi : [CRS/09_selecteurs-avances.md](#) .

Pseudo-classe (détail)

Une pseudo-classe cible un **état** ou une **position** d'un élément, pas une partie de son contenu (contrairement aux pseudo-éléments).

États courants :

```
a:hover        /* survol souris */  
a:focus        /* focus clavier ou souris */  
a:focus-visible /* focus clavier UNIQUEMENT (recommandé pour a11y) */  
a:visited      /* lien déjà visité */  
input:disabled /* input désactivé */  
input:checked  /* checkbox/radio cochée */  
input:invalid  /* input qui ne respecte pas sa validation */
```

Position dans la fratrie :

```
li:first-child /* premier <li> dans son parent */  
li:last-child  /* dernier */  
li:nth-child(odd) /* impairs */  
li:nth-child(3n+1) /* 1er, 4e, 7e... */  
li:not(.active) /* sauf ceux avec classe .active */
```

⚠ Piège

`:focus-visible` $\neq$ `:focus` . `:focus` s'active au clic souris, ce qui est souvent indésirable.
Utiliser `:focus-visible` pour ne styliser que le focus **clavier** (accessibilité).

Voir aussi : [CRS/09_selecteurs-avances.md](#) .

C. Cascade et héritage

Terme	Définition courte	Chapitre
Cascade	Algorithme qui détermine quelle règle l'emporte quand plusieurs ciblent le même élément (origine, spécificité, ordre).	X
Spécificité	Poids d'un sélecteur calculé en (ID, classes/attributs/pseudo-classes, types) . Plus élevé = priorité.	X
Héritage	Propagation automatique de certaines propriétés (color , font-family) de parent vers enfants.	X
!important	Annotation qui force une règle à l'emporter sur la cascade. À éviter sauf cas extrême.	X
Ordre source	À spécificité égale, la dernière règle déclarée dans le CSS gagne.	X

Spécificité (détail)

La spécificité se lit comme un triplet (a, b, c) :

- a = nombre d'ID dans le sélecteur.
- b = nombre de classes, attributs, pseudo-classes.
- c = nombre de types (balises) et pseudo-éléments.

Exemples :

Sélecteur	Spécificité
p	(0, 0, 1)
.btn	(0, 1, 0)
#header	(1, 0, 0)
nav .btn:hover	(0, 2, 1)
#header .nav a	(1, 1, 1)
*	(0, 0, 0) — universel = aucune spécificité

Comparaison : on compare a d'abord, puis b, puis c . (1, 0, 0) > (0, 99, 99) .

⚠ Piège

Un #header .nav a est plus spécifique qu'un .menu .nav a . Donc surcharger un style défini sur #header nécessite soit !important , soit un sélecteur encore plus spécifique. **C'est pourquoi on évite les IDs en CSS.**

Voir aussi : [CRS/10_cascade-heritage.md](#) .

Cascade (détail)

Quand plusieurs règles ciblent le même élément, la cascade applique ces critères dans l'ordre :

1. **Origine** : règles utilisateur (forcées) > règles auteur (le CSS du site) > règles agent utilisateur (defaults navigateur).
2. **!important** : une déclaration **!important** annule cet ordre.
3. **Spécificité** : à origine égale, la règle la plus spécifique gagne.
4. **Ordre source** : à spécificité égale, **la dernière déclarée** gagne.

♦ Diagnostic dans DevTools

Ouvre F12 → onglet *Elements* → panneau *Styles*. Les règles **barrées** ont été écrasées par une règle plus spécifique ou plus tardive. Survoler une règle pour voir sa spécificité.

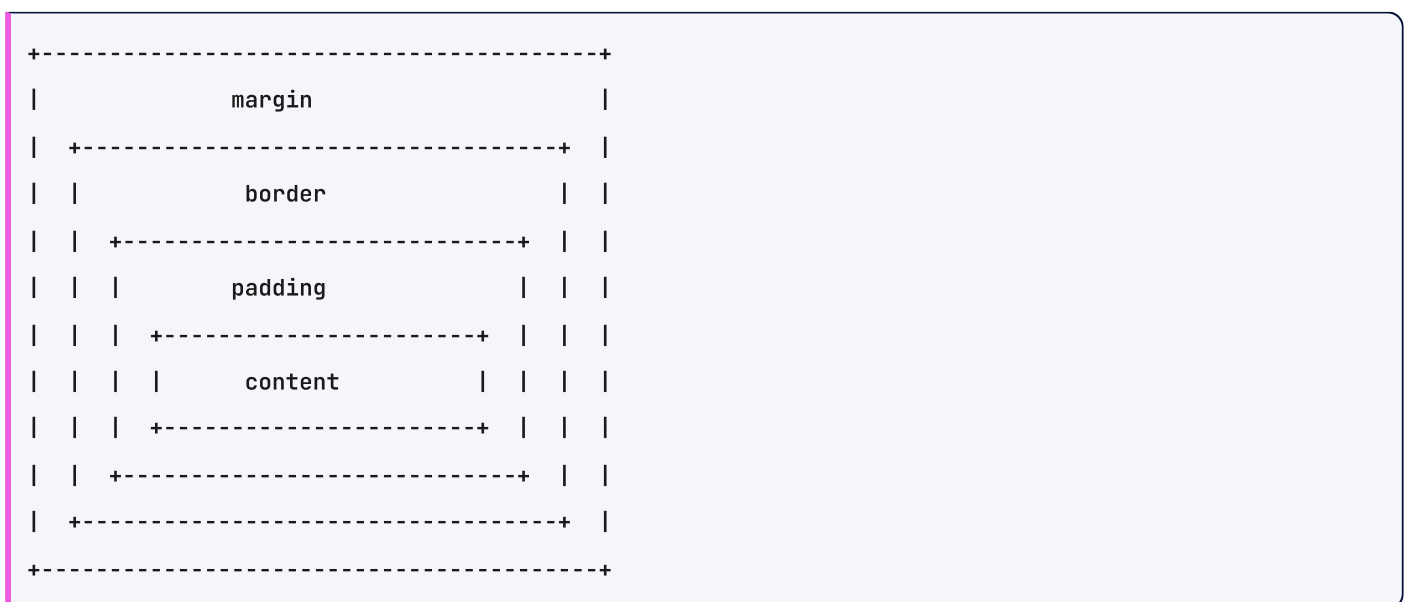
Voir aussi : [CRS/10_cascade-heritage.md](#) .

D. Box-model et layout

Terme	Définition courte	Chapitre
Box-model	Modèle qui décrit la taille d'un élément : contenu + padding + border + margin.	VI
margin	Marge extérieure entre l'élément et ses voisins.	VI
padding	Espace intérieur entre la bordure et le contenu.	VI
border	Bordure visible autour du padding.	VI
Content	Zone réelle du texte ou des enfants.	VI
box-sizing	Détermine si <code>width</code> / <code>height</code> incluent ou non le padding et la border. <code>border-box</code> recommandé.	VI
display	Type d'affichage de l'élément (<code>block</code> , <code>inline</code> , <code>inline-block</code> , <code>flex</code> , <code>grid</code> , <code>none</code>).	VI, XI, XII

Box-model (détail)

Tout élément HTML est rendu comme une **boîte** composée de 4 zones concentriques :



`width` et `height` par défaut désignent la zone **content**. Donc `width: 200px; padding: 10px; border: 5px` produit une boîte visible de `230px` de large.

`box-sizing: border-box` change ce comportement : `width` inclut padding et border. La même règle produit alors une boîte de `200px` exactement, avec un contenu de `170px`.

♦ Reset universel recommandé

```
*, *::before, *::after { box-sizing: border-box; }
```

Le calcul des largeurs devient intuitif (tu donnes la taille extérieure, le navigateur ajuste l'intérieur).

Voir aussi : [CRS/06_introduction-css.md](#) , [CRS/07_visualiser-la-structure.md](#) .

E. Flexbox et Grid

Terme	Définition courte	Chapitre
Flexbox	Modèle de layout 1 dimension (ligne OU colonne) — <code>display: flex</code> .	XI
Grid	Modèle de layout 2 dimensions (lignes ET colonnes) — <code>display: grid</code> .	XII
Axe principal	Direction du flux Flexbox (par défaut horizontal, <code>flex-direction: row</code>).	XI
Axe transversal	Direction perpendiculaire à l'axe principal.	XI
<code>fr</code>	Unité Grid pour fraction d'espace disponible (<code>1fr 2fr</code> = 1/3 et 2/3).	XII
<code>auto-fit</code> / <code>auto-fill</code>	Mots-clés Grid pour créer des colonnes automatiquement selon la place disponible.	XII
<code>minmax(min, max)</code>	Fonction Grid définissant une largeur min et max pour une colonne ou rangée.	XII
<code>gap</code>	Espace entre items Flex ou cellules Grid (remplace <code>margin</code> interne).	XI, XII

Flexbox (détail)

Activé avec `display: flex` sur le **conteneur**. Les **items** (enfants directs) se rangent automatiquement sur l'axe principal.

Propriétés conteneur principales :

```
.container {
  display: flex;
  flex-direction: row | column; /* axe principal horizontal/vertical */
  justify-content: center | space-between | flex-start; /* alignement axe principal */
  align-items: center | stretch | flex-start; /* alignement axe transversal */
  flex-wrap: nowrap | wrap; /* passer à la ligne ou non */
  gap: 1rem; /* espace entre items */
}
```

Propriétés items principales :

```
.item {  
  flex-grow: 1;      /* poids dans l'expansion (0 = pas d'expansion) */  
  flex-shrink: 1;    /* poids dans la réduction */  
  flex-basis: auto;   /* taille de base avant grow/shrink */  
  flex: 1;           /* raccourci pour grow=1, shrink=1, basis=0 */  
  align-self: center; /* override align-items pour cet item */  
}
```

Pattern courant — **centrer parfaitement** :

```
.parent {  
  display: flex;  
  justify-content: center; /* centre horizontal */  
  align-items: center;    /* centre vertical */  
  min-height: 100vh;  
}
```

⚠ Piège

Sans `flex-wrap: wrap`, les items débordent du conteneur si la somme dépasse la largeur.
Penser `flex-wrap` dès qu'on traite plusieurs cards responsive.

Voir aussi : `CRS/11_flexbox.md`. Cheatsheet locale : `cheatsheets.md`.

Grid (détail)

Activé avec `display: grid` sur le conteneur. Tu définis les **colonnes** et **rangées**, les items se placent dedans.

Propriétés conteneur principales :

```
.container {  
  display: grid;  
  grid-template-columns: 1fr 2fr 1fr; /* 3 colonnes : 1/4, 1/2, 1/4 */  
  grid-template-rows: auto 1fr auto; /* 3 rangées : header/main/footer */  
  grid-template-areas:  
    "header header header"  
    "side  main  main"  
    "footer footer footer";  
  gap: 1.5rem;  
}
```

Placement d'item par area nommée :

```
.header { grid-area: header; }  
.side   { grid-area: side; }  
.main   { grid-area: main; }  
.footer { grid-area: footer; }
```

Voir aussi : [CRS/12_grid.md](#) . Cheatsheet locale : [cheatsheets.md](#) .

auto-fit + minmax (détail)

Le pattern le plus utile de Grid pour le responsive sans media query :

```
.galerie {  
  display: grid;  
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));  
  gap: 1.5rem;  
}
```

Lecture : "crée autant de colonnes que possible, chacune entre **250px minimum** et **1 fraction de l'espace disponible maximum**". Résultat :

- Sur écran large (1200px), 4 colonnes de ~285px.
- Sur écran moyen (700px), 2 colonnes de ~340px.
- Sur mobile (350px), 1 colonne de 350px.

Sans aucune media query. C'est la magie Grid pour les galeries.

`auto-fill` est similaire mais conserve les pistes vides ; `auto-fit` les replie. `auto-fit` est presque toujours ce qu'on veut.

♦ Mnémonique

"auto-**fit** = repli (fit ensemble), auto-**fill** = remplit même avec du vide".

Voir aussi : [CRS/12_grid.md](#) .

F. Responsive

Terme	Définition courte	Chapitre
Media query	Bloc CSS <code>@media (...)</code> qui n'applique ses règles que sous conditions (largeur, hauteur, orientation, mode sombre...).	XIII
Breakpoint	Taille d'écran déclenchant un changement de layout (souvent 576px, 768px, 992px, 1200px).	XIII
Mobile-first	Écriture du CSS de base pour mobile, puis surcharges via <code>@media (min-width: ...)</code> pour écrans plus grands.	XIII
Viewport	Zone visible du navigateur. Configuré via <code><meta name="viewport"></code> .	XIII
Unité relative	Taille proportionnelle à un autre contexte : <code>em</code> (parent), <code>rem</code> (racine), <code>%</code> (parent), <code>vw</code> / <code>vh</code> (viewport).	VIII, XIII
<code>prefers-color-scheme</code>	Media query qui détecte le mode sombre/clair préféré de l'utilisateur.	XIII, XIX

Media query (détail)

Syntaxe :

```
@media (min-width: 768px) {
  /* règles appliquées dès 768px et plus */
}

@media (max-width: 575px) {
  /* règles appliquées jusqu'à 575px */
}

@media (min-width: 768px) and (orientation: landscape) {
  /* combinaisons logiques */
}
```

Breakpoints Bootstrap 5 (à connaître) :

Nom	Largeur min	Usage type
(xs) 0		Mobile portrait
sm	576px	Mobile paysage
md	768px	Tablette portrait
lg	992px	Tablette paysage / petit laptop
xl	1200px	Desktop
xxl	1400px	Grand desktop

Note balise viewport

`<meta name="viewport" content="width=device-width, initial-scale=1.0">` est **indispensable** dans `<head>` pour que le mobile prenne en compte les media queries. Sans cette ligne, le mobile zoom-dezoom comme s'il était un desktop.

Voir aussi : [CRS/13_responsive.md](#).

Mobile-first (détail)

Stratégie d'écriture du CSS qui **commence par mobile**, puis enrichit pour les écrans plus grands :

```
/* Base : mobile */
.card {
  width: 100%;
  font-size: 1rem;
}

/* À partir de tablette */
@media (min-width: 768px) {
  .card { width: 48%; }
}

/* À partir de desktop */
@media (min-width: 1200px) {
  .card { width: 30%; }
}
```

Pourquoi mobile-first ?

1. Le mobile représente > 60 % du trafic web mondial.
2. Le CSS de base reste simple (mobile = layout 1 colonne, le plus naturel).
3. Les `@media (min-width)` sont plus lisibles que des `(max-width)` chaînés.
4. Une page mobile s'affiche correctement même si les media queries ne sont pas chargées.

⚠ Anti-pattern à éviter

Écrire le CSS pour desktop puis ajouter `@media (max-width: 1199px)`, `(max-width: 991px)`, `(max-width: 767px)`, `(max-width: 575px)` — chaque media query annule la précédente. Maintenance cauchemardesque.

Voir aussi : [CRS/13_responsive.md](#).

G. Bootstrap

Terme	Définition courte	Chapitre
Framework CSS	Bibliothèque de styles et composants pré-écrits (Bootstrap, Tailwind, Bulma). Évite de réinventer la roue.	XIV
Utility-first	Approche où chaque classe fait une chose (<code>d-flex</code> , <code>p-3</code> , <code>text-center</code>). Tailwind est le plus connu. Bootstrap mixe utility + composants.	XIV
Container	Conteneur Bootstrap (<code>.container</code> , <code>.container-fluid</code>) qui centre et limite la largeur.	XV
Row	Rangée Bootstrap (<code>.row</code>) qui héberge des colonnes. Active automatiquement Flexbox.	XV
Col	Colonne Bootstrap (<code>.col</code> , <code>.col-md-6</code>) sur une grille à 12 colonnes.	XV
Composant Bootstrap	Bloc préconçu : <code>.navbar</code> , <code>.card</code> , <code>.modal</code> , <code>.alert</code> , etc. Combinaison de classes + parfois JavaScript.	XVI
CDN	<i>Content Delivery Network</i> — distribution rapide de fichiers via URL publique. Bootstrap est servi via CDN jsDelivr.	XIV
Personnalisation	Surcharge des couleurs/typo Bootstrap via variables CSS (<code>--bs-primary</code>) ou compilation Sass custom.	XVII

Utility-first (détail)

Plutôt que de définir des composants comme `.btn-primary` (qui groupe `padding` + `color` + `background` + `border`), une approche utility-first compose les styles avec des classes mono-tâche :

```

<!-- Utility-first style -->
<button class="d-inline-block px-4 py-2 bg-primary text-white border-0 rounded">
  Cliquez-moi
</button>

<!-- Composant style -->
<button class="btn btn-primary">Cliquez-moi</button>

```


Approche	Avantage	Inconvénient
Utility-first (Tailwind)	Très flexible, pas de surcharge CSS, design system cohérent	HTML chargé, courbe d'apprentissage
Composants (Bootstrap classique)	Lisible, démarrage rapide, beaux defaults	Moins flexible, risque de réécrire des classes

Bootstrap 5 **mixe les deux** : composants pour la structure (`.navbar` , `.card`) + utilities pour les ajustements (`.mt-3` , `.text-center` , `.bg-primary`).

♦ Quand choisir ?

- Site vitrine ou MVP rapide : **Bootstrap** (composants prêts).
- Design system custom, équipe maîtrisant CSS : **Tailwind** (utility-first).
- CSS custom from scratch : pour très petit site ou apprentissage.

Voir aussi : [CRS/14_introduction-bootstrap.md](#) , [CRS/18_css-ou-bootstrap.md](#) .

container / row / col (détail)

La grille Bootstrap est une **grille à 12 colonnes** activée via Flexbox sous le capot. Structure obligatoire :

```
<div class="container">           <!-- centre + limite largeur -->
  <div class="row">                 <!-- une rangée -->
    <div class="col-md-6">A</div>    <!-- 6/12 sur md+ -->
    <div class="col-md-6">B</div>    <!-- 6/12 sur md+ -->
  </div>
</div>
```

Suffixes responsive : `col-{breakpoint}-{1-12}` . Sans suffixe, `col` = colonnes flexibles équiréparties.

```
<div class="row">
  <div class="col-12 col-md-6 col-lg-4">A</div> <!-- mobile 100%, tablet 50%, desktop 33% -->
</div>
```

⚠ Piège

Si tu mets une `.col` hors d'une `.row` , ça **ne marche pas** : pas de Flexbox parent. Et une `.row` hors d'un `.container` peut déborder horizontalement.

Voir aussi : [CRS/15_grid-et-utilities.md](#) . Cheatsheet locale : [cheatsheets.md](#) .

Index alphabétique

Terme	Section
Accessibilité	A. HTML sémantique
ARIA	A. HTML sémantique
Attribut	A. HTML sémantique
Axe principal	E. Flexbox et Grid
Axe transversal	E. Flexbox et Grid
<code>auto-fit</code> / <code>auto-fill</code>	E. Flexbox et Grid
Balise	A. HTML sémantique
Balise sémantique	A. HTML sémantique
<code>border</code>	D. Box-model et layout
Box-model	D. Box-model et layout
<code>box-sizing</code>	D. Box-model et layout
Breakpoint	F. Responsive
Cascade	C. Cascade et héritage
CDN	G. Bootstrap
Classe (CSS)	B. CSS — fondamentaux
Col (Bootstrap)	G. Bootstrap
Combinateur	B. CSS — fondamentaux
Composant Bootstrap	G. Bootstrap
Container (Bootstrap)	G. Bootstrap
Content	D. Box-model et layout
Déclaration (CSS)	B. CSS — fondamentaux
<code>display</code>	D. Box-model et layout
Élément (HTML)	A. HTML sémantique
Flexbox	E. Flexbox et Grid
Framework CSS	G. Bootstrap
<code>fr</code> (Grid)	E. Flexbox et Grid
<code>gap</code>	E. Flexbox et Grid
Grid	E. Flexbox et Grid
Héritage	C. Cascade et héritage
ID (CSS)	B. CSS — fondamentaux
<code>!important</code>	C. Cascade et héritage

margin	D. Box-model et layout
Media query	F. Responsive
minmax()	E. Flexbox et Grid
Mobile-first	F. Responsive
Ordre source	C. Cascade et héritage
padding	D. Box-model et layout
Personnalisation Bootstrap	G. Bootstrap
prefers-color-scheme	F. Responsive
Propriété (CSS)	B. CSS — fondamentaux
Pseudo-classe	B. CSS — fondamentaux
Pseudo-élément	B. CSS — fondamentaux
Règle CSS	B. CSS — fondamentaux
Row (Bootstrap)	G. Bootstrap
Sélecteur	B. CSS — fondamentaux
Spécificité	C. Cascade et héritage
Unité relative	F. Responsive
Utility-first	G. Bootstrap
Validation native	A. HTML sémantique
Valeur (CSS)	B. CSS — fondamentaux
Viewport	F. Responsive

Pour aller plus loin

- MDN HTML reference — developer.mozilla.org/fr/docs/Web/HTML/Element
- MDN CSS reference — developer.mozilla.org/fr/docs/Web/CSS/Reference
- CSS-Tricks Almanac — css-tricks.com/almanac/ — référence des propriétés CSS avec exemples
- A Complete Guide to Flexbox — css-tricks.com/snippets/css/a-guide-to-flexbox/
- A Complete Guide to CSS Grid — css-tricks.com/snippets/css/complete-guide-grid/
- Bootstrap 5 docs — getbootstrap.com/docs/5.3/
- WebAIM Contrast Checker — webaim.org/resources/contrastchecker/
- Can I Use — caniuse.com — compatibilité navigateurs